

Mini SIEM:

Automated Security Event Detection and Analysis

Group 6

Prepared By:

Ahmed Abd Elnasser	21133233
Shahd Ahmed Mohammed Gamel	21125230
Androw Akmal Fadeil	21131006
Esmail Ahmed Esmail Hassan	21130651
Omar Mahmoud Fawzy	21121252

System Analysis & Design

May 2026

1. Problem Statement & Objectives

1.1 Problem Statement

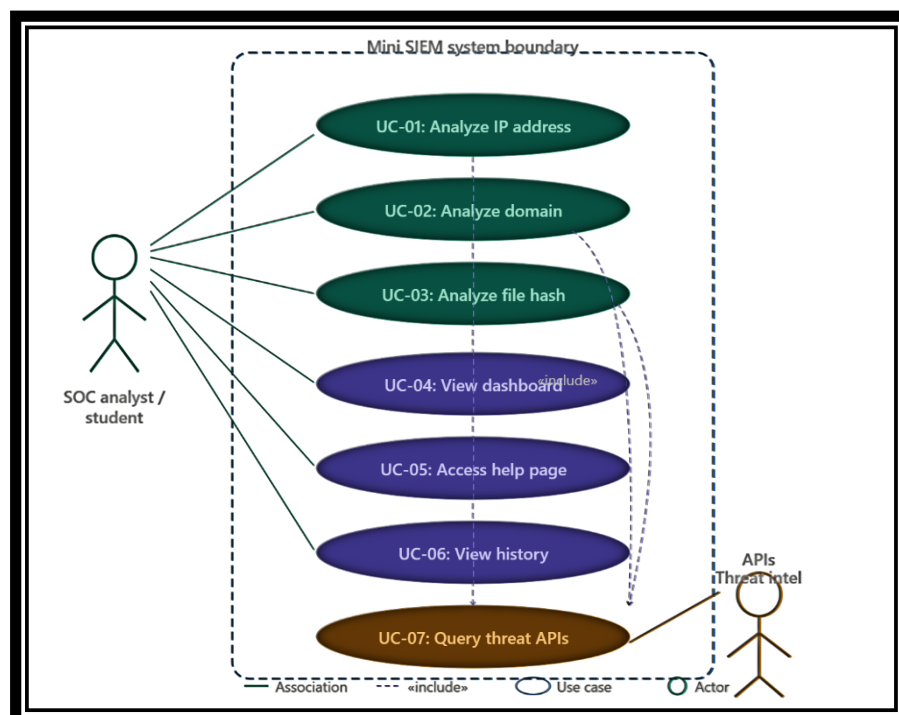
Security Operations Center (SOC) analysts and small organizations face a critical operational challenge: the manual, repetitive, and time-consuming process of verifying suspicious Indicators of Compromise (IOCs) across multiple threat intelligence platforms. Analysts must individually query platforms such as VirusTotal, AbuseIPDB, and AlienVault OTX, manually correlate the results, and form a verdict a process that is inefficient, error-prone, and inaccessible to those without enterprise-grade tooling.

1.2 Project Objectives

1. Design and implement a web-based Mini SIEM platform that automates IOC analysis across multiple threat intelligence APIs.
2. Build a unified analysis engine that aggregates and normalizes results from VirusTotal, AbuseIPDB, and AlienVault OTX.
3. Implement a rule-based detection engine with ML-assisted scoring to classify threat severity automatically.
4. Deliver a user-friendly web dashboard that visualizes threat intelligence results clearly and intuitively.
5. Provide an educational help section to support junior analysts and cybersecurity students.
6. Produce a fully documented, modular, and maintainable codebase following professional software engineering standards.

1.3 Use Case Diagram & Descriptions

The following Use Case Diagram illustrates the interactions between the system actors and the core functionalities of the Mini SIEM system.



1.3.1 System Actors

1- SOC Analyst / Student (Primary Actor): The user responsible for inputting data, monitoring the dashboard, and investigating specific security events.

2- Threat Intel APIs (Secondary Actor): External systems (e.g., VirusTotal) that provide the necessary security reputation data upon request from the Mini SIEM.

1.3.2 Key Use Case Descriptions

Based on the system architecture:

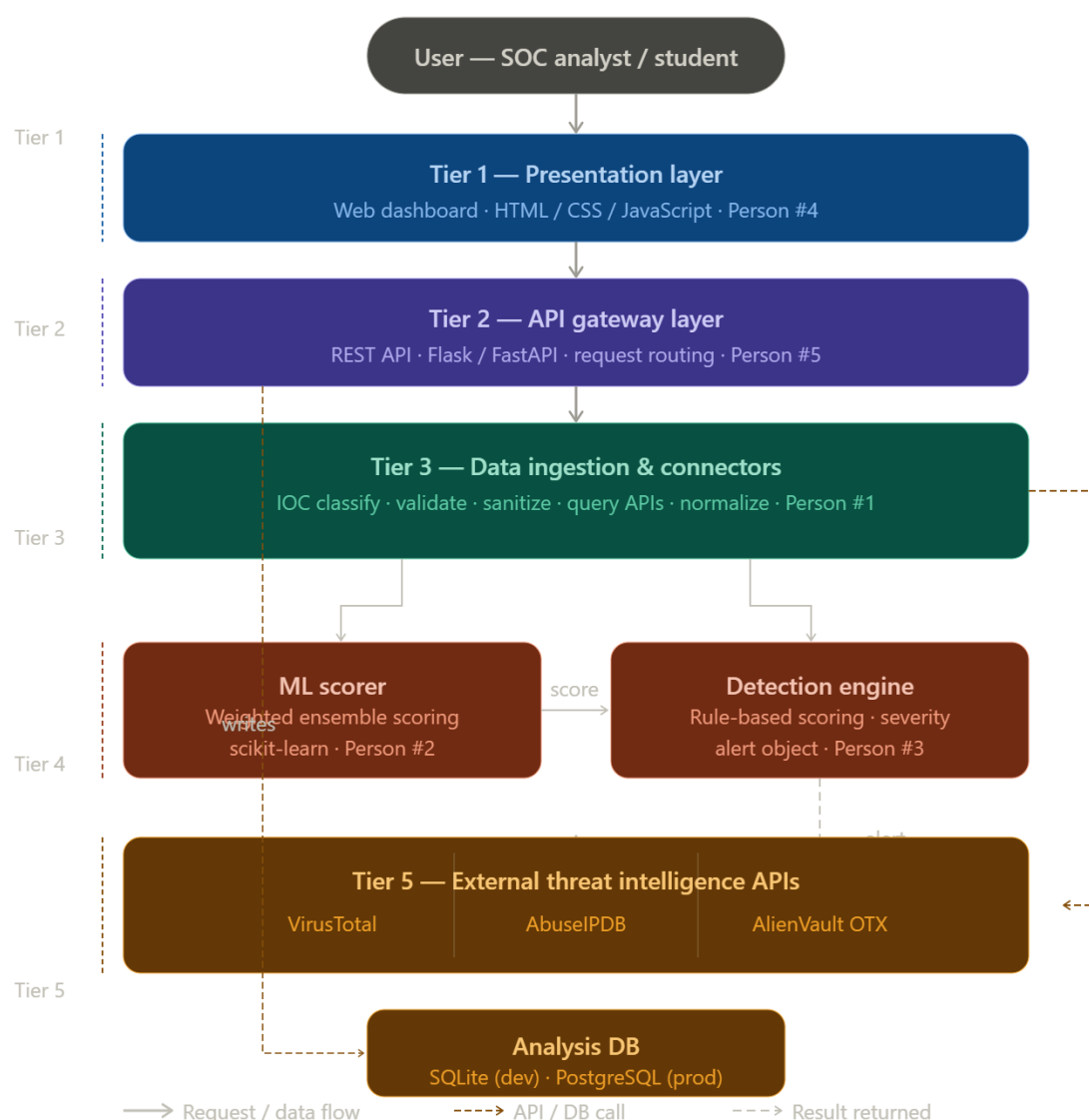
- **UC-01 to UC-03 (Targeted Analysis):** Allows the analyst to perform specific lookups for IP addresses, Domains, and File Hashes to determine their threat level.
- **UC-04 (View Dashboard):** The central hub for monitoring live alerts; this process includes the automated querying of threat APIs to ensure data is current.
- **UC-05 (Access Help Page):** Allows any user, particularly cybersecurity students and junior analysts, to navigate to the instructional section of the dashboard for guidance on interpreting threat results and understanding IOC types.
- **UC-06 (View History):** Enables the analyst to retrieve and review previous investigation results for audit and reporting purposes.
- **UC-07 (Query Threat APIs):** An automated background process where the system communicates with external intelligence providers to enrich the gathered data.

1.4 Functional & Non-Functional Requirements

To ensure system success, the following requirements have been defined:

- **Functional Requirements:** The system must accept IP addresses, domains, and file hashes as user input, automatically classify and validate them, query VirusTotal, AbuseIPDB, and AlienVault OTX, apply rule-based and ML-assisted scoring, assign a severity level (SAFE to CRITICAL), and display results on a web dashboard with color-coded indicators.
- **Non-Functional Requirements:** The system must protect all API keys via environment variables (Security), return analysis results within 15 seconds (Performance), function correctly when one API is unavailable (Reliability), and provide a beginner-friendly interface with a help page (Usability).

1.5 Software Architecture



1.5.1 Architectural Components

The Mini SIEM is organized into five tiers, each owned by one team member and connected through clearly defined interfaces.

1. **Tier 1 Presentation Layer:** The web dashboard is the only part the user interacts with. It provides an IOC input form, a color-coded results view showing severity and per-API scores, an analysis history page, and a help page for new users. Built with HTML, CSS, and JavaScript.
2. **Tier 2 API Gateway Layer :** Acts as the communication bridge between the dashboard and all backend components. It exposes REST API endpoints, routes user input to the ingestion layer, returns the final verdict to the dashboard as a standardized JSON response, and writes results to the database. Built with Python Flask / FastAPI.
3. **Tier 3 Data Ingestion & Connectors :** The entry point of the analysis pipeline. It automatically classifies the user's input as an IP address, domain, or file hash, validates and sanitizes it, queries the applicable threat intelligence APIs, handles errors and rate limits gracefully, and normalizes all responses into a single unified data object passed to Tier 4. Built with Python and the Requests library.

4. **Tier 4 Intelligence Layer** : Two parallel components both receive the normalized data object from Tier 3.
 - **ML Scorer** : applies a weighted scoring formula combining scores from all three APIs into a single confidence score (0–100).
 - **Detection Engine** : applies rule-based thresholds to assign a severity level (SAFE / LOW / MEDIUM / HIGH / CRITICAL) and produces the final alert object containing the IOC, verdict, score, and triggered rules.
5. **Tier 5 External Threat Intelligence APIs** : Three public APIs queried by Tier 3. All API keys are stored in environment variables never hardcoded.

API	Supported IOC Types	Data Returned
VirusTotal	IP, domain, hash	Engine scan results from 70+ AV engines
AbuseIPDB	IP only	Abuse confidence score, report count
AlienVault OTX	IP, domain, hash	Threat pulse count, malware families

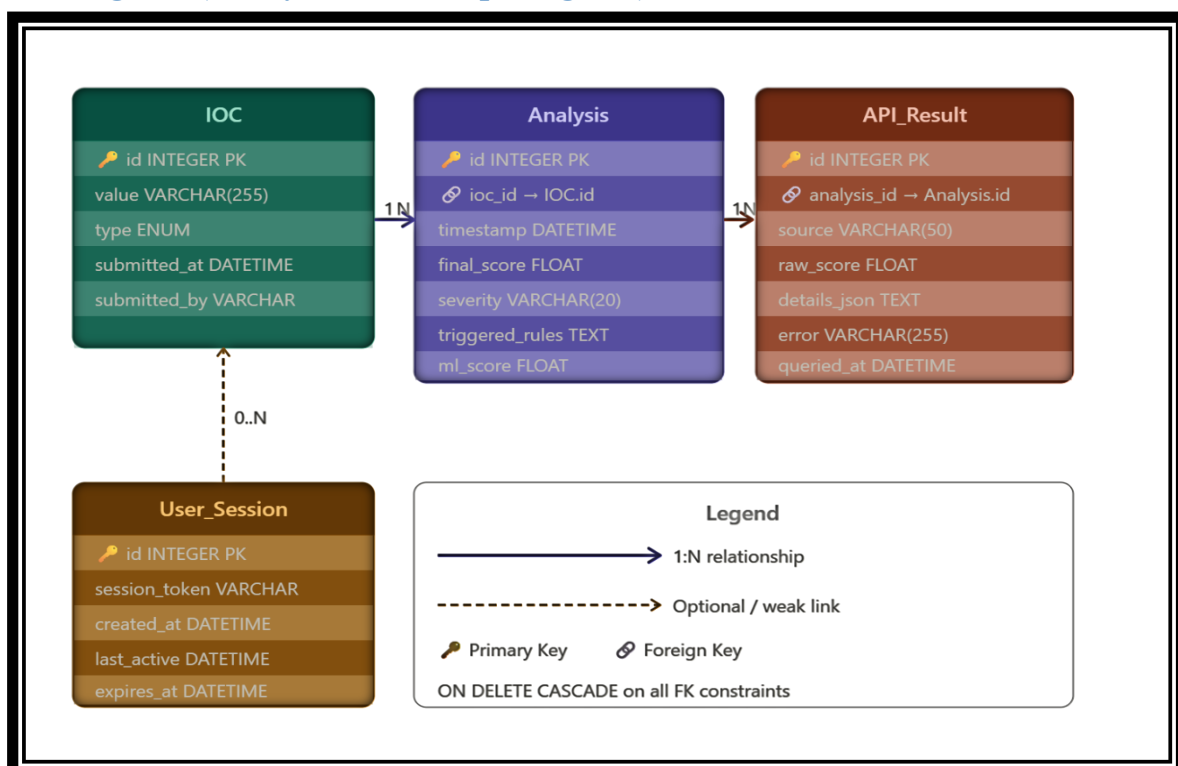
Analysis DB : SQLite in development, PostgreSQL in production. Written to by the API Layer only.

1.5.2 Architectural Style

The Mini SIEM follows a **Layered (N-Tier) Architecture**. Each tier has one responsibility and communicates only with the tier directly adjacent to it — the dashboard never calls APIs directly, the ingestion layer never assigns severity, and the detection engine never talks to the database. This strict separation allows all five team members to develop their components in parallel without blocking each other. A shared data contract — the normalized JSON object produced by Tier 3 — is agreed upon before coding begins and serves as the integration point between all components.

2. Database Design & Data Modeling

2.1 ER Diagram (Entity-Relationship Diagram)



The system consists of four primary entities:

- **IOC (Indicator of Compromise):** Stores the unique technical indicators identified by the system.
 - Attributes: id (Primary Key), value (string), type (ip/domain/hash), submitted_at (datetime), and submitted_by (string).
- **Analysis:** Captures the core results of the intelligent processing layer.
 - Attributes: id (Primary Key), ioc_id (Foreign Key), timestamp (datetime), final_score (float), severity (string), and triggered_rules (text).
- **API Result:** Stores the raw and enriched data retrieved from external threat intelligence sources.
 - Attributes: id (Primary Key), analysis_id (Foreign Key), source (VT/Abuse/OTX), raw_score (float), details_json (text), and error (string/null).
- **User Session:** Manages active user interactions and authentication tokens.
 - Attributes: id (Primary Key), session_token (string), and created_at (datetime).

The diagram defines a hierarchical data flow through the following relationships:

- **IOC to Analysis (1 → N):** One IOC can undergo multiple analyses over time to track changes in threat status.
- **Analysis to API Result (1 → N):** A single analysis event triggers multiple external API lookups (e.g., querying VirusTotal and AbuseIPDB simultaneously).
- **User Session to IOC (Optional):** An optional relationship indicating which user session initiated the submission of a specific indicator.

2.2 Logical & Physical Schema

1. Table: ioc Stores the technical indicators submitted for analysis.

- id: Primary Key (PK) to uniquely identify each record.
- value: The raw submitted data — an IP address, domain name, or file hash.
- type: Classifies the input as ip, domain, md5, sha1, or sha256.
- submitted_at: Automatically records the submission timestamp.

2. Table: analysis The central table recording the outcome of each detection run.

- id: Primary Key (PK).
- ioc_id: Foreign Key (FK) linking this result to its parent ioc record (1:N).
- final_score & ml_score: Numeric risk values from the Detection Engine and ML Scorer.
- severity: Human-readable verdict ranging from SAFE to CRITICAL.
- triggered_rules: JSON string listing every detection rule that fired.

3. Table: api_result Stores the raw response from each external threat intelligence provider.

- id: Primary Key (PK).
- analysis_id: Foreign Key (FK) linking this record to its parent analysis (1:N).
- source: Identifies the provider — VirusTotal, AbuseIPDB, or AlienVault OTX.
- raw_score: The primary numeric score extracted from the API response.
- details_json: Full raw API response stored for auditing purposes.
- error: Records any failure reason if the API call did not succeed.

4. Table: user_session Manages user access and activity tracking.

- id: Primary Key (PK).
- session_token: A unique string used to identify and authenticate the session.
- created_at & last_active: Track when the session started and was last used.
- expires_at: Optional expiry timestamp — sessions past this value are rejected.

user_session			
INTEGER	id	PK	AUTO_INCREMENT
VARCHAR_255	session_token		UNIQUE NOT NULL
DATETIME	created_at		DEFAULT NOW()
DATETIME	last_active		DEFAULT NOW()
DATETIME	expires_at		nullable



ioc			
INTEGER	id	PK	AUTO_INCREMENT
VARCHAR_255	value		NOT NULL
ENUM	type		ip, domain, md5, sha1, sha256
DATETIME	submitted_at		DEFAULT NOW()
VARCHAR_100	submitted_by		nullable



analysis			
INTEGER	id	PK	AUTO_INCREMENT
INTEGER	ioc_id	FK	NOT NULL CASCADE
DATETIME	timestamp		DEFAULT NOW()
FLOAT	final_score		NOT NULL
VARCHAR_20	severity		SAFE to CRITICAL
TEXT	triggered_rules		JSON string
FLOAT	ml_score		nullable

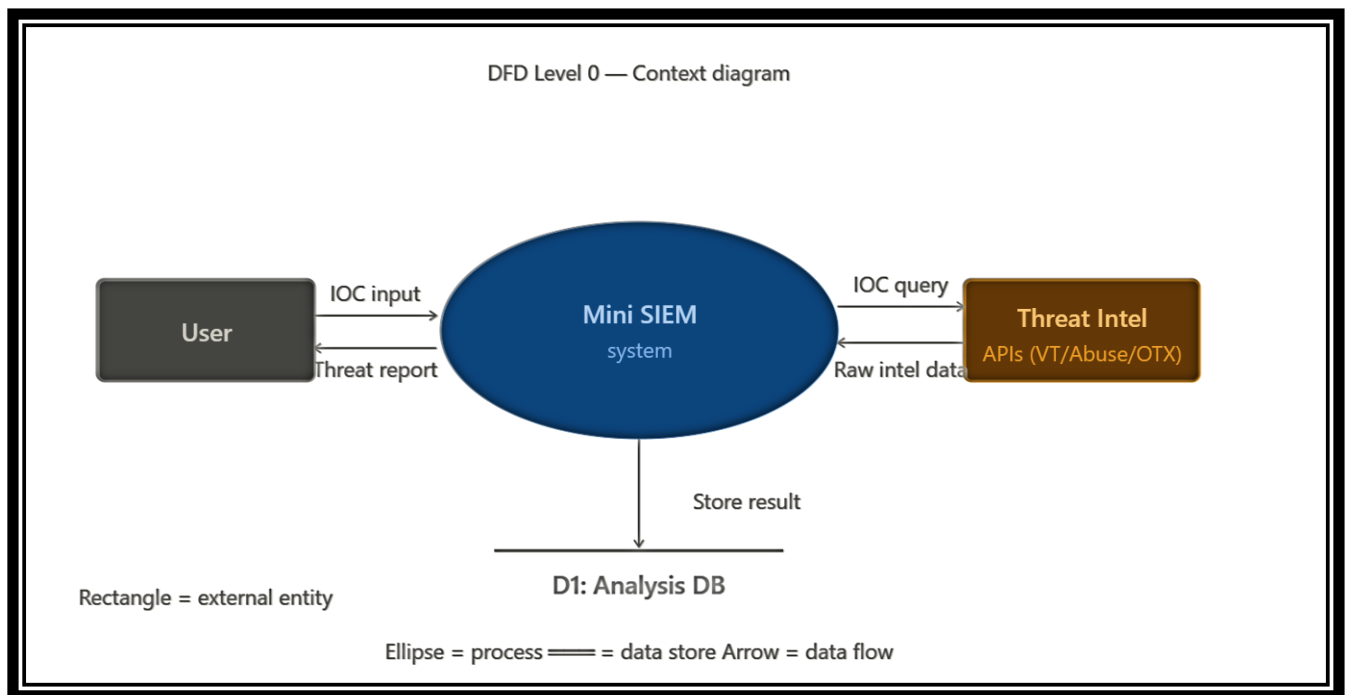


api_result			
INTEGER	id	PK	AUTO_INCREMENT
INTEGER	analysis_id	FK	NOT NULL CASCADE
VARCHAR_50	source		VT, AbuseIPDB, OTX
FLOAT	raw_score		nullable
TEXT	details_json		nullable
VARCHAR_255	error		nullable
DATETIME	queried_at		DEFAULT NOW()

3. Data Flow & System Behavior



3.1 DFD (Data Flow Diagram)



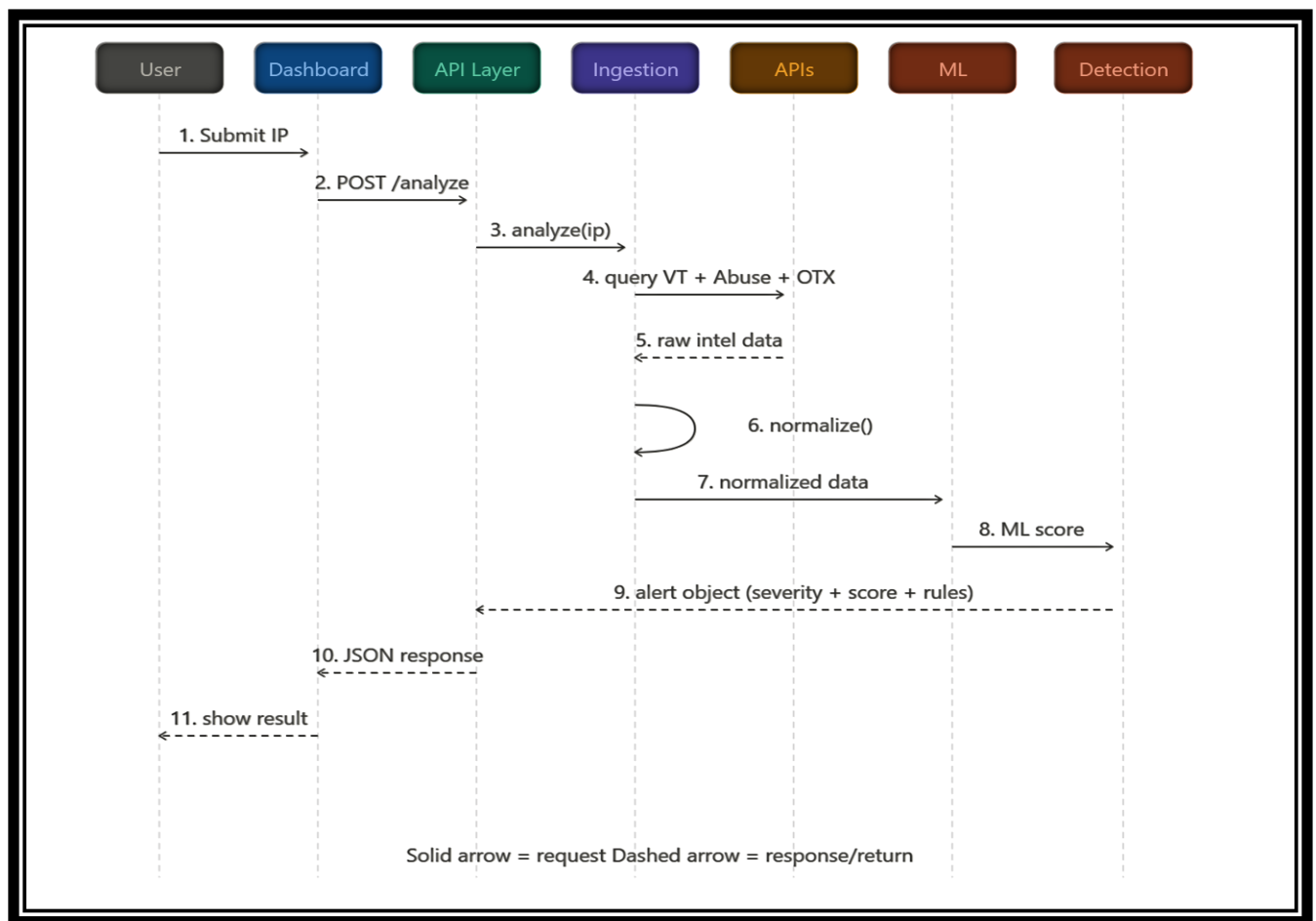
3.1.1 Data Flow Description (DFD Level 0 — Context Level)

At the context level, the Mini SIEM system has two primary external entities: the End User and the External Threat Intelligence APIs. The End User submits IOCs to the system and receives threat reports. The External APIs receive query requests from the system and return raw intelligence data.

3.1.2 DFD Level 1 — Internal Processes

Process	Inputs	Outputs
Input Classification & Validation	Raw user input (string)	Validated IOC + type label
API Query Execution	Validated IOC + type	Raw API responses (JSON)
Response Normalization	Raw API JSON responses	Unified normalized data object
ML Scoring	Normalized data	ML confidence score (0-100)
Rule-Based Detection	Normalized data + ML score	Final verdict + severity + triggered rules
API Response Formatting	Final verdict	Standardized JSON response
Dashboard Rendering	JSON response	Visual threat report to user

3.2. Sequence Diagram Description



The sequence diagram illustrates the chronological flow of messages and actions required to process a single security indicator:

- 1. Initiation:** The User submits an IP address through the Dashboard.
- 2. Request Routing:** The Dashboard sends a POST request (/analyze) to the API Layer, which then passes the analysis command to the Ingestion component.
- 3. Threat Enrichment:** The Ingestion component queries external APIs (VirusTotal, AbuseIPDB, and OTX) and receives the raw threat intelligence data.
- 4. Normalization:** The Ingestion component performs a self-loop action to normalize() the raw data into a standard format.
- 5. Intelligent Analysis:** The normalized data is sent to the ML layer, which calculates an ML Score.
- 6. Detection & Scoring:** The Detection engine processes the ML score and generates a comprehensive Alert Object containing the severity level, final score, and triggered rules.
- 7. Response Delivery:** The alert object is returned through the API Layer as a JSON response to the Dashboard, which then displays the final result to the User.

3.3 Activity Diagram : The following activity diagram outlines the operational flow of the Mini SIEM from initial user input to final result display.

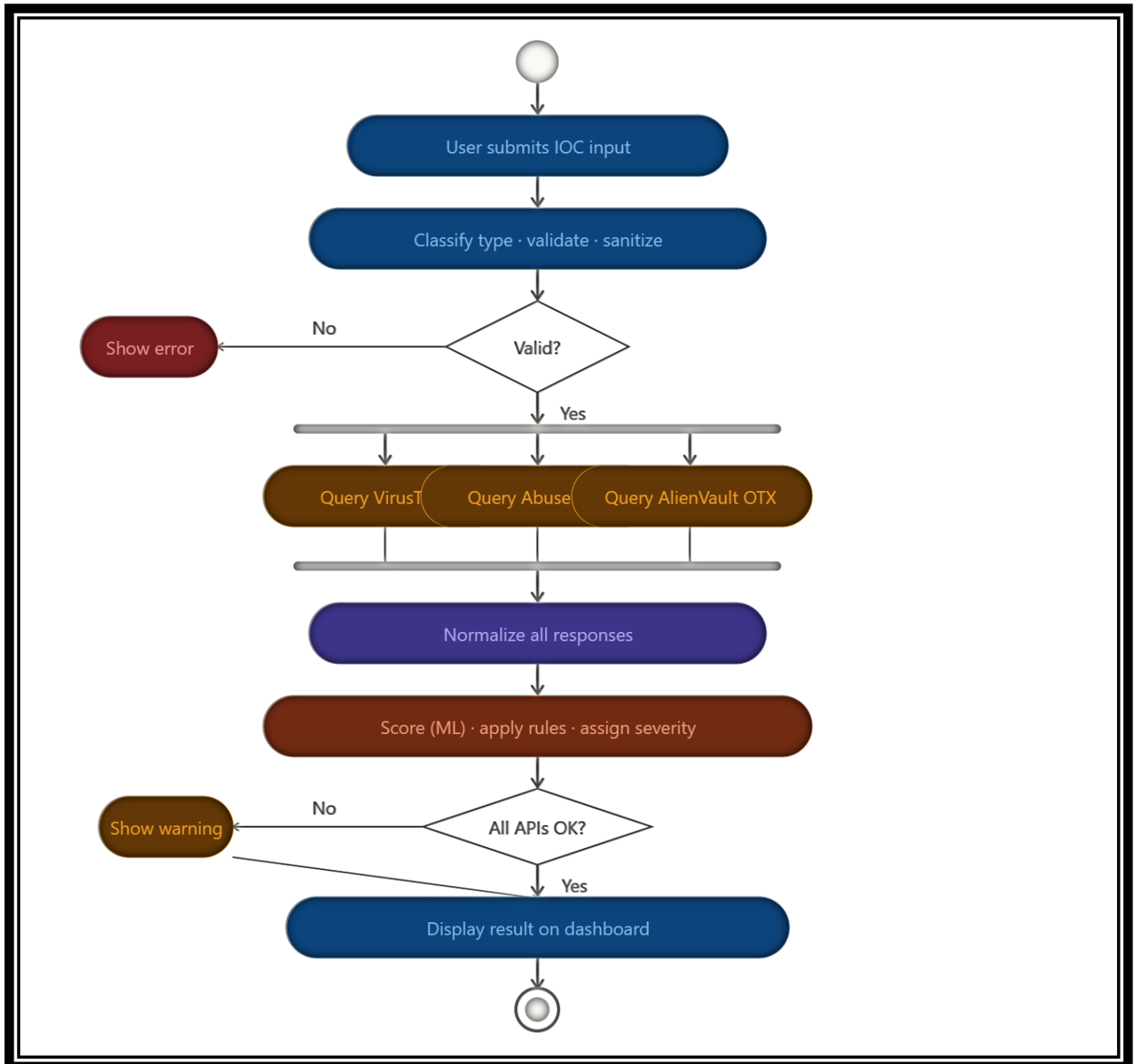


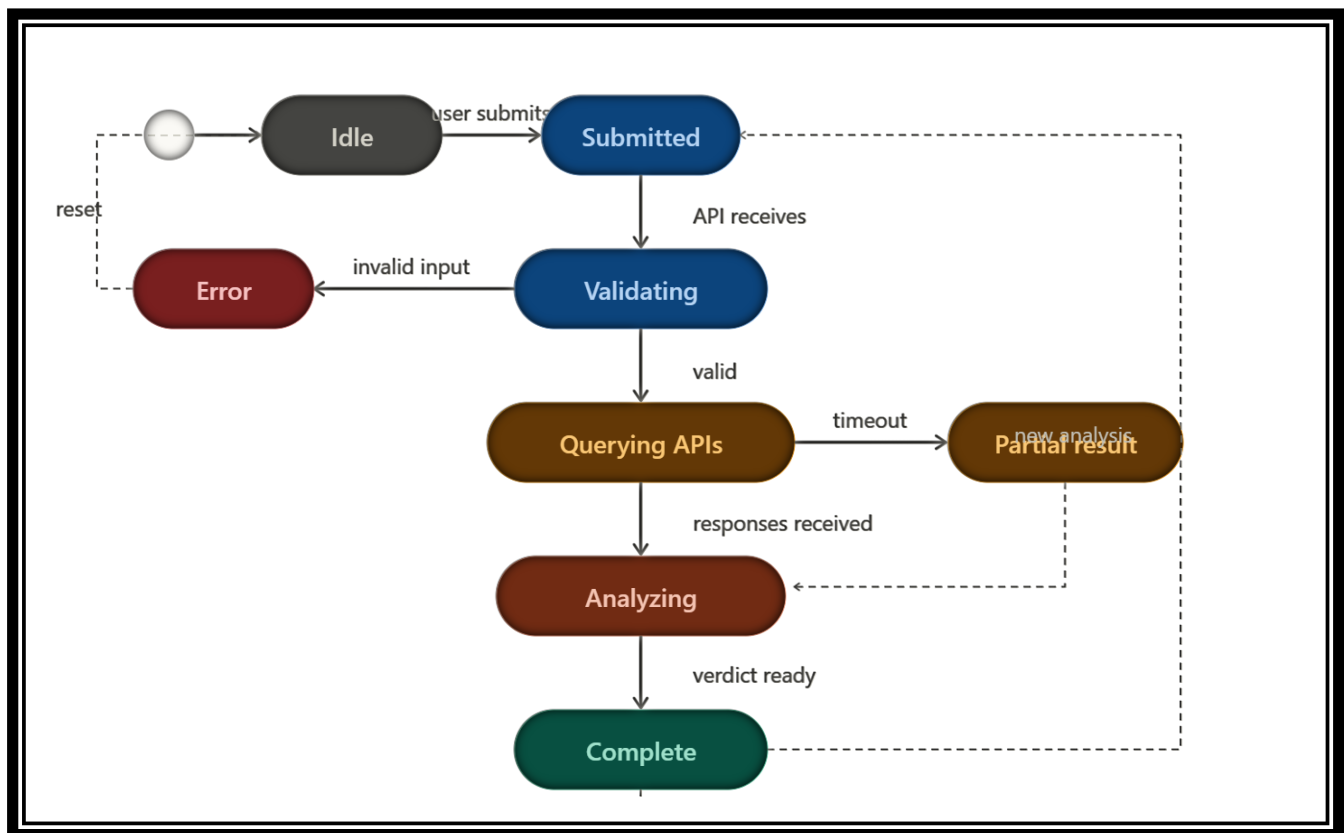
Diagram outlines the operational flow of the system from the initial user input to the final display of results:

- **Input Validation Phase:**
 - The process begins when the user submits an IOC input.
 - The system performs classification (IP, Domain, or Hash), validation, and sanitization.
 - Decision Point: If the input is invalid, the system triggers a "Show error" action.
- **Parallel Processing (Threat Enrichment):**
 - If valid, the system forks into parallel activities, querying multiple APIs simultaneously: VirusTotal, AbuseIPDB, and AlienVault OTX.

- **Data Processing Phase:**
 - The system joins the parallel flows to normalize all responses into a single format.
 - The Machine Learning (ML) scoring is applied, followed by rule application and severity assignment.
- **Final Output Phase:**
 - Decision Point: The system checks if all APIs responded correctly. If not, a "Show warning" is triggered.
 - The final step is to Display the result on the dashboard.

3.4 State Diagram :

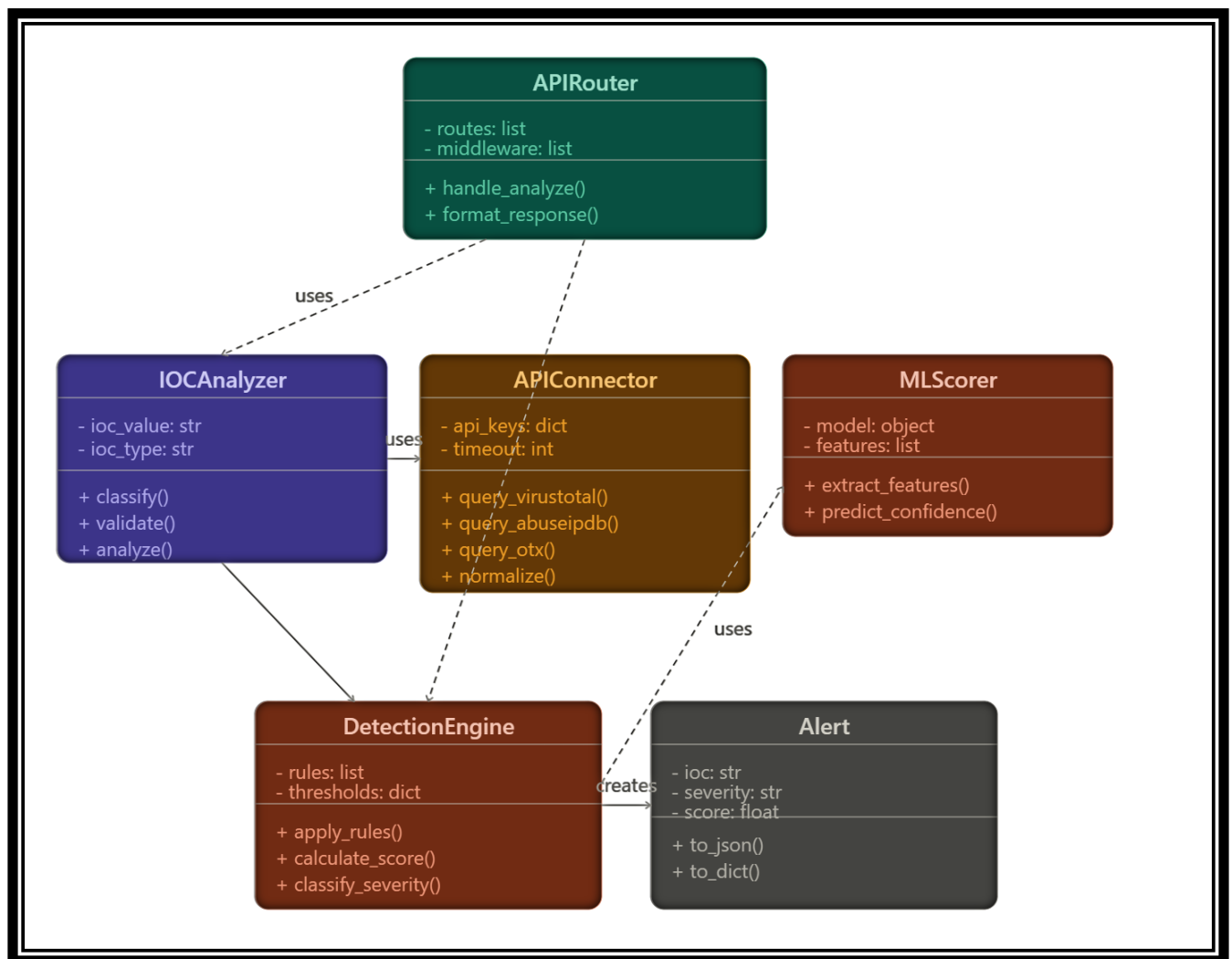
Shows the different states an object can be in and what triggers it to change state. Like a traffic light , it's always in one state (red/yellow/green) and an event causes it to switch.



The State Diagram illustrates the operational lifecycle of a threat analysis request:

- **Initial States:** The system transitions from Idle to Submitted upon receiving a user request, then moves to Validating.
- **Error Handling:** If validation fails, the system enters the Error state and resets.
- **Processing:** Validated inputs trigger the Querying APIs state. Timeouts result in a Partial Result, while successful responses lead to the Analyzing state for ML scoring.
- **Completion:** Once a verdict is ready, the system enters the Complete state, displaying results before allowing a new submission.

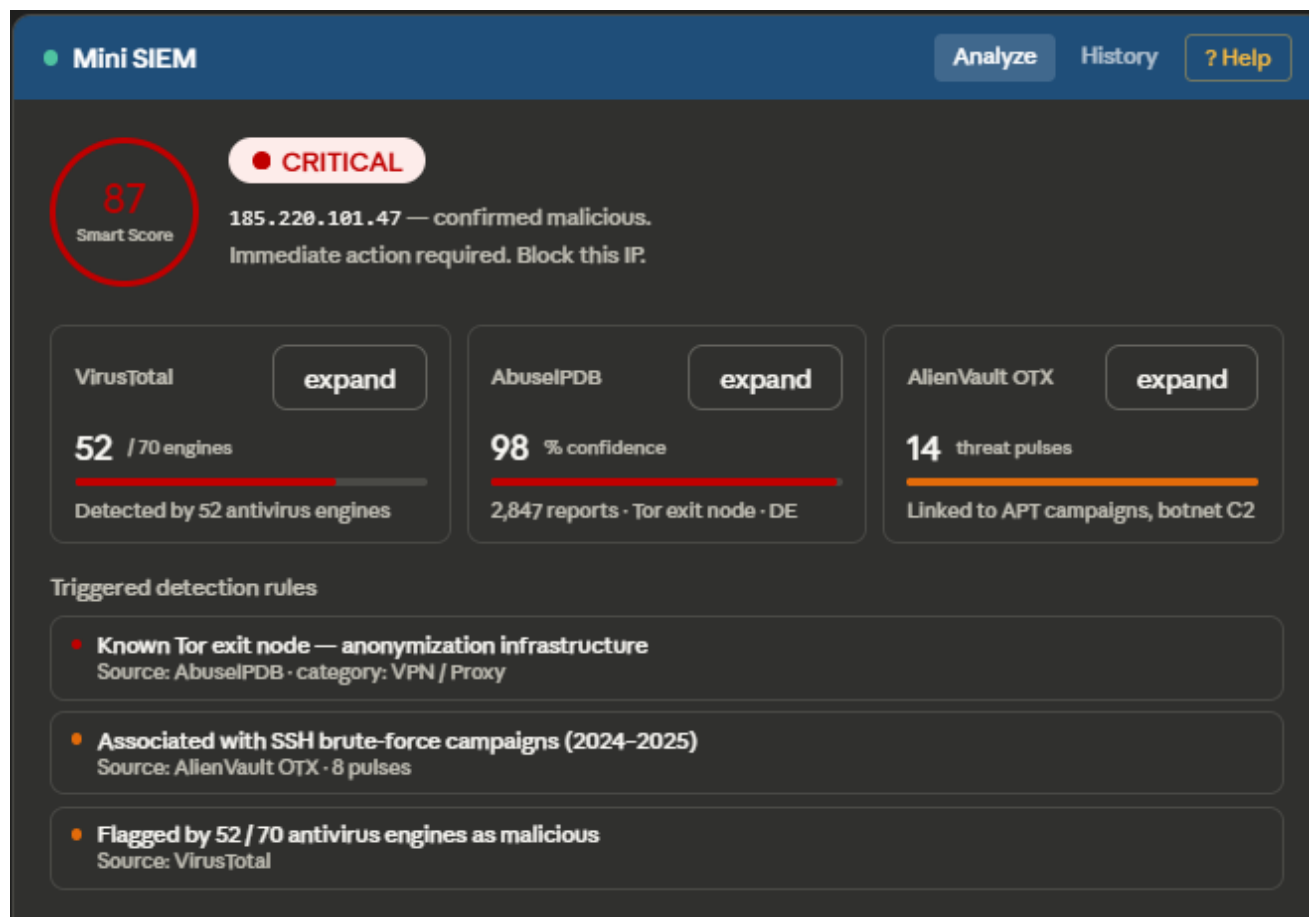
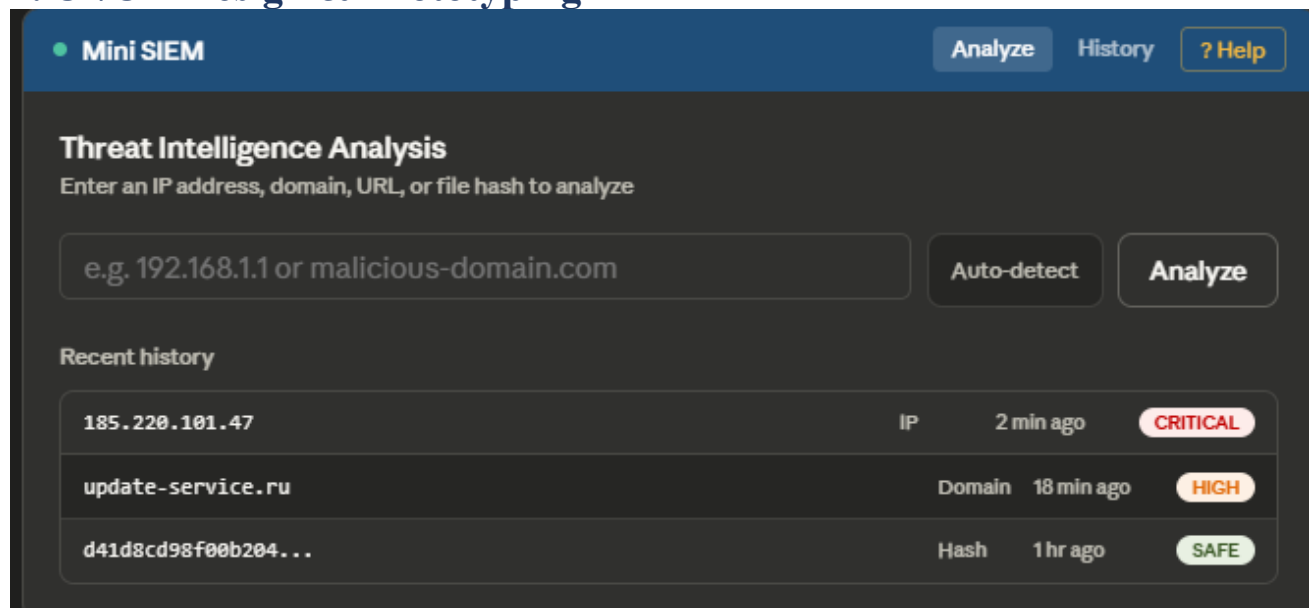
3.5 Class Diagram



The Class Diagram defines the object-oriented structure of the Mini-SIEM, detailing the attributes and methods for each core module:

- **APIRouter:** Acts as the entry point, managing system routes and middleware. It handles incoming requests via `handle_analyze()` and formats the final output.
- **IOCAnalyzer:** Responsible for the initial processing of technical indicators. It contains methods to `classify()`, `validate()`, and `analyze()` the specific IOC type (IP, domain, or hash).
- **APIConnector:** Manages external communications. It stores `api_keys` and implements methods to query specific providers like VirusTotal, AbuseIPDB, and OTX, followed by a `normalize()` function.
- **MLScorer:** The intelligent layer of the system. It uses a trained model to `extract_features()` and `predict_confidence()` scores for detected anomalies.
- **DetectionEngine:** The decision-making hub. It applies security rules and thresholds to `calculate_score()` and `classify_severity()`.
- **Alert:** A data object created by the Detection Engine. It encapsulates the final severity and score, providing methods to export data to JSON or dictionary formats for the frontend.

4. UI/UX Design & Prototyping



4.1 Wireframes & Mockups

The interface is designed to provide a "Single Pane of Glass" view for the analyst, focusing on clarity and rapid response.

- **Search & Input Interface:** The main entry screen features a clean, centralized input bar with an "Auto-detect" feature to simplify the submission of IPs, domains, or hashes. A "Recent History" table provides immediate access to prior investigations with color-coded severity badges.
- **Analysis Results Dashboard:** Upon execution, the system renders a high-density report. Key components include a Smart Score gauge, expanded data cards for each integrated API (VirusTotal, AbuseIPDB, OTX), and a detailed list of Triggered Detection Rules to justify the severity level.

4.2 UI/UX Guidelines

The design follows industry-standard security interface principles to ensure effective data interpretation:

- **Color Scheme:** A Dark Mode aesthetic is utilized to reduce eye strain for analysts. A semantic color palette is used for risk levels: Red (Critical/Malicious), Orange (High/Suspicious), and Green (Safe).
- **Typography:** A highly readable Sans-serif font is used throughout to maintain clarity in technical reports and logs.
- **Hierarchy:** Critical data, such as the "Smart Score" and "Severity Badge," is placed at the top-left to follow the natural F-pattern eye movement of users.
- **Accessibility:** All interactive elements, such as "Expand" buttons and navigation tabs, feature high-contrast outlines and clear labeling to support efficient navigation during high-stress incident responses.

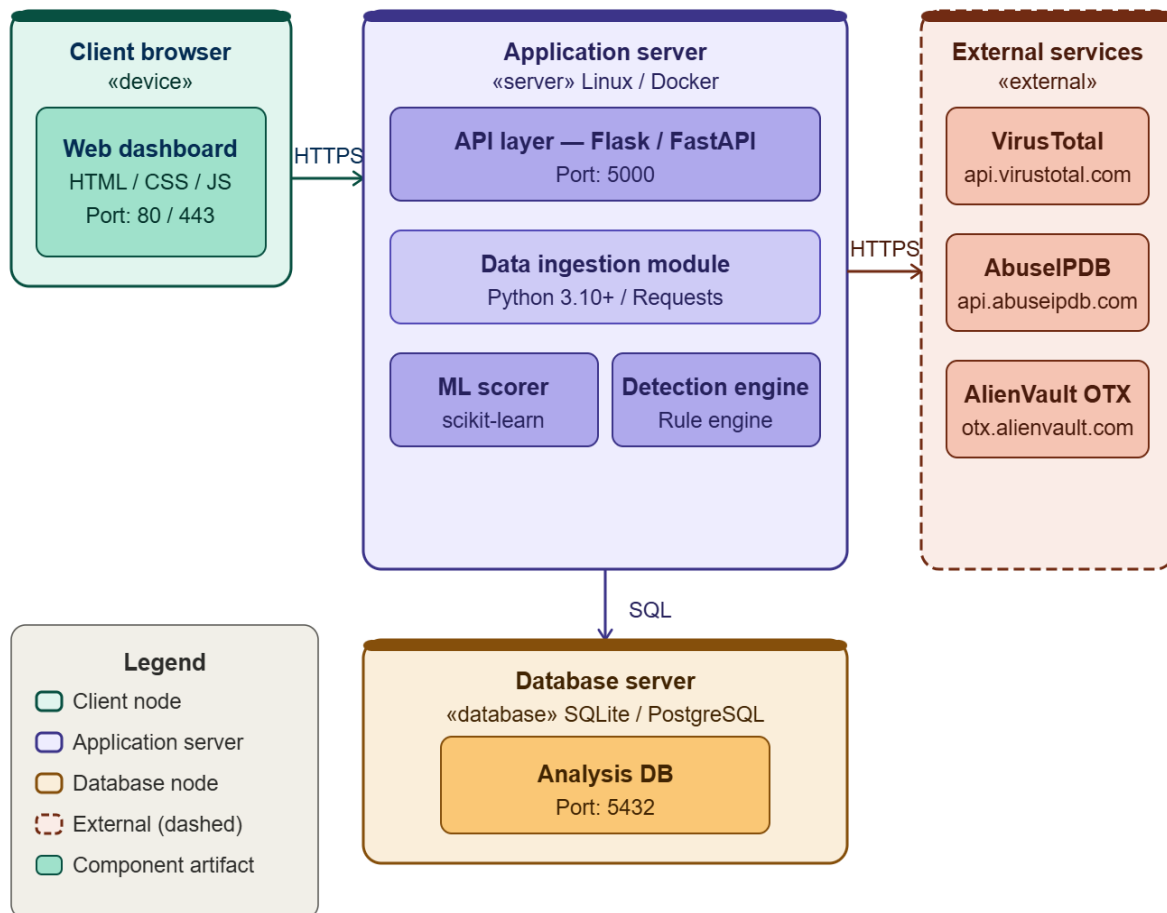
5. System Deployment & Integration

5.1 Technology Stack

The system is built using a modern, scalable tech stack to ensure high performance and seamless integration between components:

- **Frontend:** **React.js** or **Vue.js** for a responsive, real-time dashboard interface.
- **Backend:** **Python (Flask or FastAPI)**, chosen for its extensive libraries in cybersecurity, machine learning, and API handling.
- **Database:** **PostgreSQL** or **MySQL** for structured storage of IOCs and analysis history.
- **Machine Learning:** **Scikit-learn** for implementing the weighted ensemble scoring model.
- **Deployment:** **Docker** for containerization and **GitHub** for version control and CI/CD.

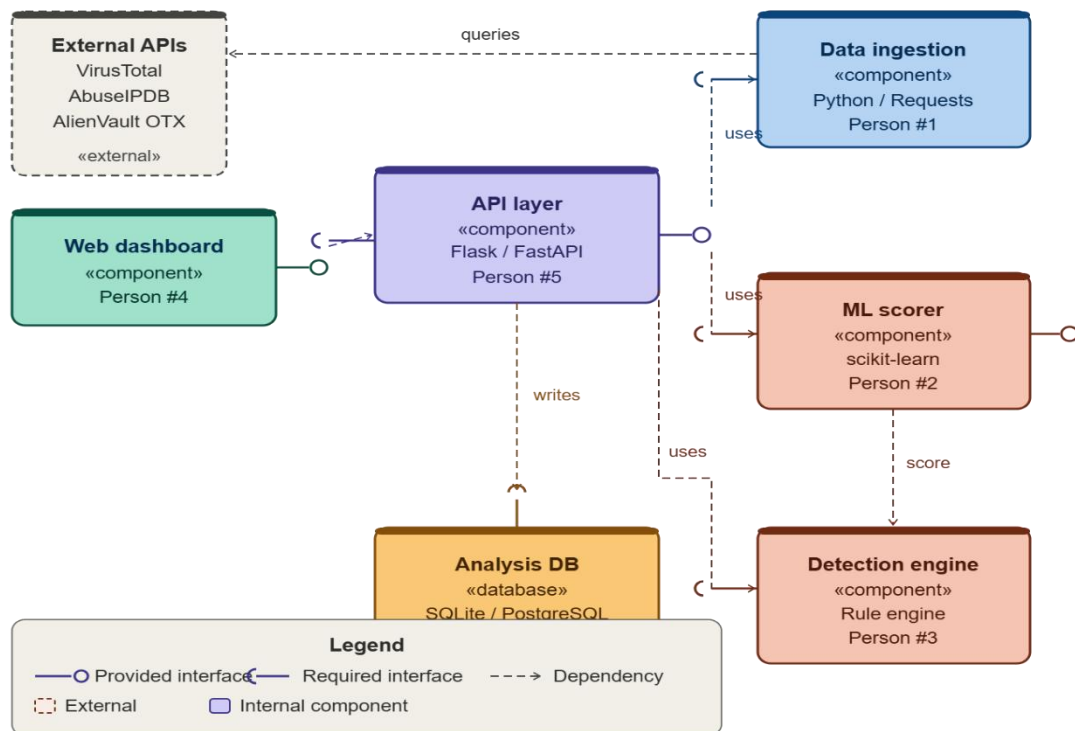
5.2 Deployment Diagram



The Mini-SIEM uses a distributed architecture across four tiers:

- **Client Tier:** A **Web Dashboard** (HTML/CSS/JS) communicating via **HTTPS** on ports **80/443**.
- **Application Tier:** A **Linux/Docker** server hosting the **API Layer** (Flask/FastAPI), **Data Ingestion** (Python), and the **ML/Detection engines**.
- **Database Tier:** A **PostgreSQL/SQLite** server hosting the **Analysis DB** on port **5432**.
- **External Tier:** Integration with **Threat Intelligence APIs** (VirusTotal, AbuseIPDB, OTX) via secure outbound requests.

5.3 Component Diagram



The Mini-SIEM is structured into modular components with clearly defined dependencies and interfaces:

- **Web Dashboard** : The primary user interface that requires a connection to the API Layer to function.
- **API Layer** : The central hub (Flask/FastAPI) that provides the interface for the dashboard while managing dependencies on the database and core processing modules.
- **Data Ingestion** : A Python-based component responsible for external communication with **VirusTotal**, **AbuseIPDB**, and **AlienVault OTX**.
- **Intelligence Tier** : Consists of the **ML Scorer** (scikit-learn) and the **Detection Engine** (Rule Engine). These modules process normalized data to generate the final security verdict.
- **Analysis DB**: A persistent storage component (SQLite/PostgreSQL) that the API Layer depends on for logging and history retrieval